



Scale: what this server costs at library sizes nobody had tried

Every measurement in this repository until 2026-09-06 was taken against a library of **23 songs and 238 KB**. [docs/DEPLOY-VAULT2.md](#) said so plainly under "what this still does not prove" — *"one library, 22 songs, 224 KB. Nothing here says anything about a real collection's size or scan time."* This document is that gap, closed as far as synthetic data can close it, and honest about the part it cannot.

Why synthetic, and what that costs

A real library means community-charted songs, and community charts bundle the copyrighted recording. This project's permanent non-goals include distributing copyrighted audio, and a test corpus that cannot be rebuilt on another machine is not much of a test corpus. So [cmd/mkscale](#) **generates** a library: real parseable WAV audio from a sine wave, a real 1×1 PNG, a real [.chart](#), varied metadata drawn from word lists so sorting and search have something to do. It is deterministic — the same [-seed](#) and [-n](#) give the same library byte for byte — so two runs on two machines are comparable.

```
mkscale -out <dir> -n 10000 -audio-bytes 65536 -zip-percent 10 -seed 1
```

It is deliberately **not** [cmd/mkcorpus](#). That writes 23 songs which each probe one parsing decision and every one of them matters; this writes thousands of ordinary songs whose only job is volume. Keeping them apart means a scale experiment cannot quietly change what the oracle measures.

The generator's first version was wrong, and the server is what caught it. Note values were derived from the song name as [rune % 5](#), so two names of equal length whose characters happened to agree mod 5 — ["... 4"](#) and ["... 9"](#), for instance — charted identically. 10,000 songs produced **9,733 distinct charts** and 267 shared-chart groups. That was a generator bug rather than a server one, and the fix was to write the index into the chart. It is recorded here because it is the honest form of a scale number: *a synthetic library can be less varied than its song count suggests*, and anyone reading the tables below should want to know that the [distinct_charts](#) column is checked, not assumed.

Hardware

ENG-1: Intel Core Ultra 7 155H (16 cores / 22 threads), 96 GB RAM, SK hynix NVMe SSD, Windows 11 Pro, Go 1.27.0 windows/amd64. Numbers on a Raspberry Pi will be very different; see "What this still does not prove".

Count: index time and memory are linear in songs

Songs	Files	Library bytes	Index time	RSS	Distinct charts	Problems
1,000	4,109	67,552,723	594 ms	20.1 MB	1,000	0
10,000	41,435	682,215,531	5.833 s	79.1 MB	10,000	0
31,109	128,799	2,121,663,333	17.703 s	215.2 MB	31,109	0

Per song that is 0.594 ms, 0.583 ms and 0.569 ms — **flat, and very slightly better as the library grows**. Memory fits **13 MB + 6.5 KB per song** across the whole range: the increments are 6.56 KB/song from 1k to 10k and 6.45 KB/song from 10k to 31k.

Those two constants are the useful output of this document, because they extrapolate:

Library	Predicted index	Predicted RSS
50,000	~29 s	~340 MB
100,000	~58 s	~665 MB

That last row is a real constraint on goal #1. A Raspberry Pi 3B+ has 1 GB of RAM and would not hold a 100,000-song catalog with room to serve from it; a 4 GB or 8 GB Pi 4/5 would. The catalog is held in memory by design (ADR-002), which is the right trade at the sizes a person actually has, and this is the number that says where "actually has" ends.

The 31,109-song row is a **truncated** generation rather than a round number, and saying so matters: a background 50,000-song generator was killed when the bridge call that started it timed out, leaving exactly songs 0–31,108 complete. The library is still internally consistent — the generator finishes each song before starting the next — so it is a valid measurement, but it is not a number anyone chose.

One number in here was wrong before it was re-measured

An earlier 10,000-song run reported **5.926 s**, then **9.317 s**, then **5.833 s**. The middle one was taken while a second generator was writing 2 GB to the same SSD. Nothing about the server changed. A performance number taken on a busy machine is not a performance number, and the only reason this was caught is that the run was repeated when the machine was quiet.

Bytes: packing streams, and the cache bound holds under real pressure

The count axis uses 64 KB of audio per song, which is roughly a thousandth of a real one. The byte axis uses **5 MB per song**, and it is a different question: not "how fast can we walk a directory tree" but "what does it cost to pack and serve a gigabyte".

200 songs, 830 files, **1,065,423,677 bytes** on disk, indexed in 1.7 s.

Pack cache bound	Songs served	Bytes served	Per song	Throughput	Cache at end	RSS
2 GB (default, never reached)	200 / 200	1,171,944,253	95.3 ms	58.6 MB/s	200 files, 1.17 GB	16.5 MB
100 MB (11× oversubscribed)	200 / 200	1,171,944,253	89.2 ms	62.7 MB/s	17 files, 102,249,669 B	16.5 MB

Three things there, in order of importance:

- **Resident memory is 16.5 MB while serving 1.17 GB, and identical under both bounds.** The server streams packs from disk rather than assembling them in memory, which is a property the code was written to have and which nothing had previously measured. A server that buffered would have shown it here.
- **The bound held at 11× oversubscription and every song was still delivered.** 200 of 200, 1.17 GB, with a cache that never exceeded 102 MB. This is the same property that `pack_cache_max` shipped without — six red-proofed tests and it did not work — so it is worth re-measuring rather than trusting.
- **Eviction cost nothing measurable.** The tight run was marginally *faster*, which is not a real speedup — it is the two runs being within noise of each other because both are bound by reading and hashing a gigabyte, not by cache bookkeeping.

What this still does not prove

- **Nothing here is a real chart.** Every song is generated: uniform sizes, one chart format, clean UTF-8, no missing stems, no odd encodings, no 400 MB stem sets, no hand-authored `song.ini` weirdness. `cmd/mkcorpus` probes those shapes deliberately, but only 23 of them. **A real library's variety is untested at scale.**
- **The oracle has never run at scale.** The project's standard is that every song YARG rejects should be one this scanner independently flags, and at 23 songs that means two refusals. A real library of thousands would produce a real distribution of YARG's rejection reasons, and that is the measurement most likely to find a category we do not flag. It needs real community charts and a YARG install pointed at them.

- **These are x86-64 numbers on a fast NVMe laptop.** Index time is dominated by walking a directory tree and reading small files, so a Pi on an SD card will be far slower and the ratio is not predictable from here. The memory constant should carry across; the time constant should not be assumed to.
- **~~Concurrency is untested.~~ Closed 2026-09-07, and it found a real defect** — an intermittent 404 for a song that exists, when eviction removed an archive between the handler taking its path and opening it. See [ADR-002](#), "Concurrency". What is now measured: the thundering herd collapses to one pack; 7,680 requests across 64 clients against a single-archive cache bound returned byte-identical archives with zero mismatches; and the bound is a high-water target rather than a hard cap, overshooting by one to three archives as concurrency rises.

Still untested: concurrency *at scale*. These runs used 40 songs, not 31,109, and the clients were goroutines on the same machine rather than real ones over a network.

What the concurrency work cost, and three lessons about instruments

The defect above took four attempts to pin down, and every wrong turn was the measurement rather than the server. That is worth recording, because this project's whole method is to trust measurement over reasoning, and a measurement can be wrong in ways that reasoning is not.

A regression test has to be tuned against the defect, not written and hoped over. The first version — cache bounded to two archives, one pass over the library — **passed on the broken code**. A test that catches a bug one run in three is decoration. One archive and three rounds fails every time the defect is reintroduced, and the numbers are in the test with a note saying why they are those numbers.

The harness blamed the wrong component. Using `http.Get` opened a fresh connection per request; at 7,680 requests roughly one run in six then failed with a few responses whose status was `0`. That is the test machine running out of sockets, not the server losing a song — but the failure counter reported both identically, and the noise was briefly written up as a *second* defect inside the pack cache. It was not. Pooling connections removed it, and a build with the supposed fix disabled then passed five runs out of five. `fetch` now reports transport errors as a separate category for exactly this reason.

Reasoning about a platform is not measuring it. The tidy fix for the remaining window was to hold a handle on the temp file and rename underneath it. That was argued from Go's Windows share flags and is simply wrong: `os.Rename` fails while the source is open, and it broke every serial request the moment it was tried. One test run replaced a confident paragraph.